

Assignment NumPy 2

****Question 1:**** What is broadcasting in NumPy? State the broadcasting rules (comparing shapes right-to-left) and explain, with a suitable example, how a 1D array can be added to a 2D array without explicitly copying data.

****->****

Broadcasting in NumPy is a feature that allows NumPy to perform operations on arrays with different shapes without explicitly copying or reshaping the data.

Broadcasting Rules:

When comparing array shapes, NumPy compares them from right to left. Two dimensions are compatible if:

They are equal, or
One of them is 1.

If the dimensions are not compatible, NumPy raises a broadcasting error.

****Question 2:**** Write a NumPy program that creates a 2D array of shape (3, 4) using `np.arange()` and `reshape()`. Compute the sum along `axis=0` and `axis=1` using `np.sum()`. Explain, in your own words, what `axis=0` and `axis=1` mean and why the two results are different.

****->**** `import numpy as np`

Create a 2D array of shape (3, 4)
`arr = np.arange(1, 13).reshape(3, 4)`

`print("Array:")`
`print(arr)`

Sum along `axis=0`
`sum_axis_0 = np.sum(arr, axis=0)`

Sum along `axis=1`
`sum_axis_1 = np.sum(arr, axis=1)`

```
print("Sum along axis=0:", sum_axis_0)
```

```
print("Sum along axis=1:", sum_axis_1)
```

****Question 3:**** What is the purpose of `np.random.seed()`? Write a program that uses `np.random.rand()`, `np.random.randn()`, and `np.random.randint()` to generate three different types of random data, and explain the distribution each function draws from.

****->**** `np.random.seed()` is used to make random numbers reproducible. When the same seed value is used, NumPy generates the same sequence of random numbers every time the program runs.

```
import numpy as np
```

```
    Set the seed for reproducible results
np.random.seed(42)
```

```
    Generate random data
```

```
random_uniform = np.random.rand(3)
```

```
random_normal = np.random.randn(3)
```

```
random_integer = np.random.randint(1, 10, 3)
```

```
print("Random numbers using rand():", random_uniform)
```

```
print("Random numbers using randn():", random_normal)
```

```
print("Random integers:", random_integer)
```

****Question 4:**** Look at the code below and answer the questions:

```
import numpy as np
arr = np.arange(12)
reshaped = arr.reshape(3, 4)
flat = reshaped.flatten()
reshaped[0, 0] = 99
print(reshaped)
print(flat)
```

What will be printed? Why does changing `reshaped[0, 0]` not affect `flat`, even though `reshape()` itself does not copy the underlying data? Also explain the difference between `np.hstack()` and `np.vstack()` with a short example.

```
**->**import numpy as np
```

```
arr = np.arange(12)
reshaped = arr.reshape(3, 4)
flat = reshaped.flatten()
```

```
reshaped[0, 0] = 99
```

```
print(reshaped)
print(flat)
` ``
```

****Output:****

```
[[99  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
` ``
```

****Explanation:****

``reshape()`` changes the shape of an array without copying the underlying data when possible. Therefore, ``reshaped`` shares the same data with ``arr``.

However, ``flatten()`` always creates a ****new copy**** of the array. Therefore, changing ``reshaped[0, 0]`` to ``99`` does not affect ``flat``, because ``flat`` has its own separate copy of the data.

Difference between ``np.hstack()`` and ``np.vstack()``

``np.hstack()`` joins arrays ****horizontally (column-wise)****, while ``np.vstack()`` joins arrays ****vertically (row-wise)****.

```
a = np.array([[1, 2],
              [3, 4]])
```

```
b = np.array([[5, 6],
```

```
[7, 8]])
```

```
print(np.hstack((a, b)))
print(np.vstack((a, b)))
```
```

**\*\*Output:\*\***

```
[[1 2 5 6]
 [3 4 7 8]]
```

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```
```

Therefore:

```
* `np.hstack()` → joins arrays horizontally
* `np.vstack()` → joins arrays vertically
```

****Question 5:**** What is a Pandas DataFrame, and how does it differ from a Pandas Series? Load the built-in 'tips' dataset using Seaborn, and use head(), shape, and info() to describe what each of these methods tells you about the dataset.

****->****

A ****Pandas DataFrame**** is a two-dimensional, tabular data structure consisting of rows and columns. It can store different types of data in different columns.

A ****Pandas Series**** is a one-dimensional data structure that contains a single column of data along with an index.

****Example:****

```
import pandas as pd
import seaborn as sns
```

```
Load the built-in tips dataset
tips = sns.load_dataset("tips")
```

Display the first 5 rows

```
print(tips.head())
```

Display the shape of the dataset

```
print(tips.shape)
```

Display information about the dataset

```
print(tips.info())
```

Explanation of the methods:

* **head()**: Displays the first 5 rows of the DataFrame by default. It helps us quickly understand the structure and values of the dataset.

* **shape**: Returns the number of rows and columns in the dataset in the form `(rows, columns)`. The `tips` dataset has **244 rows and 7 columns**, so its shape is `(244, 7)`.

* **info()**: Provides a summary of the DataFrame, including the number of rows, column names, data types, and non-null values. It also shows the memory usage of the DataFrame.

In short: A **Series** is one-dimensional, while a **DataFrame** is two-dimensional and can contain multiple Series as columns.

Question 6: Explain the difference between `.loc[]` and `.iloc[]` in Pandas, including whether each is inclusive or

exclusive of the end index. Using the Titanic dataset

`(sns.load_dataset("titanic"))`, write code that selects rows 0 to 5 and the columns 'sex', 'age', and 'fare' using `.loc[]`, and the first 5 rows with the 3rd and 4th columns using `.iloc[]`.

-> Answer:

In Pandas, **`.loc[]`** and **`.iloc[]`** are used to select rows and columns from a DataFrame.

* **`.loc[]`** selects data using **labels** (row/column names).

* **`.iloc[]`** selects data using **integer positions** (index numbers).

Difference in end index

* **`.loc[]` is inclusive** of the end index.
* **`.iloc[]` is exclusive** of the end index, similar to normal Python slicing.

```
import pandas as pd
import seaborn as sns
```

```
Load Titanic dataset
titanic = sns.load_dataset("titanic")
```

```
Using .loc[]: rows 0 to 5 and selected columns
loc_data = titanic.loc[0:5, ['sex', 'age', 'fare']]
print("Using loc:")
print(loc_data)
```

```
Using .iloc[]: first 5 rows and 3rd and 4th columns
iloc_data = titanic.iloc[0:5, 2:4]
print("\nUsing iloc:")
print(iloc_data)
```

For ``.loc[0:5, ...]``, rows **0, 1, 2, 3, 4, and 5** are selected because ``.loc[]`` includes the ending label ``5``.

For ``.iloc[0:5, 2:4]``, rows **0 to 4** are selected because ``.iloc[]`` excludes the ending position ``5``. Columns at positions **2 and 3** (the 3rd and 4th columns) are selected because the ending position ``4`` is excluded.

****In short:****

Method	Selection based on	End index
-----	-----	-----
<code>`.loc[]`</code>	Labels	**Inclusive**
<code>`.iloc[]`</code>	Integer positions	**Exclusive**

****Question 7:**** Look at the code below and answer the questions:

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("titanic")
result = df[(df['age'] > 18) & (df['survived'] == 1)]
print(result.shape)
```

What does the boolean expression inside `df[...]` select? Why are the parentheses around each condition mandatory here, and what would `'&'` be replaced with if you wanted rows matching EITHER condition instead of BOTH?

```
**->** import pandas as pd
import seaborn as sns

df = sns.load_dataset("titanic")

result = df[(df['age'] > 18) & (df['survived'] == 1)]

print(result.shape)
```

The boolean expression inside ``df[...]'`` selects ****rows where both conditions are true****:

1. ``df['age'] > 18`` → selects passengers whose age is greater than 18.
2. ``df['survived'] == 1`` → selects passengers who survived.

The ``&`` operator means ****AND****, so only rows satisfying ****both conditions**** are selected.

The parentheses around each condition are mandatory because Pandas requires each comparison to be evaluated separately before applying the ``&`` operator. Without parentheses, Python's operator precedence can cause the expression to be interpreted incorrectly.

If we want rows matching ****EITHER condition****, we replace ``&`` with the ****`|` (OR) operator****:

```
result = df[(df['age'] > 18) | (df['survived'] == 1)]
```

* `&` → ****AND**** → both conditions must be true.
* `|` → ****OR**** → at least one condition must be true.
* Parentheses → required around each condition when using `&` or `|` in Pandas.

****Question 8:**** What are the two major strategies for handling missing values in a Pandas DataFrame? Write code to:

(a) drop rows where the 'age' column is missing, and (b) fill missing 'age' values with the median and missing 'embarked' values with the mode. Briefly explain when you would choose dropping over filling.

****->****

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("titanic")

result = df[(df['age'] > 18) & (df['survived'] == 1)]

print(result.shape)
```

The boolean expression inside `df[...]` selects ****rows where both conditions are true****:

1. `df['age'] > 18` → selects passengers whose age is greater than 18.
2. `df['survived'] == 1` → selects passengers who survived.

The `&` operator means ****AND****, so only rows satisfying ****both conditions**** are selected.

The parentheses around each condition are mandatory because Pandas requires each comparison to be evaluated separately before applying the `&` operator. Without parentheses, Python's operator precedence can cause the expression to be interpreted incorrectly.

If we want rows matching ****EITHER condition****, we replace `&` with the ****`|` (OR) operator****:

```
result = df[(df['age'] > 18) | (df['survived'] == 1)]
```

****Question 9:**** Explain the IQR (Interquartile Range) method for detecting outliers, including how the lower and upper bounds are calculated. Write a Python program that computes Q1, Q3, and IQR for the 'fare' column of the Titanic dataset, identifies the outliers, and then caps (Winsorizes) them instead of removing them.

****->****

The ****IQR (Interquartile Range)**** method is used to detect outliers in a dataset.

```
* **Q1 (First Quartile)** = 25th percentile
* **Q3 (Third Quartile)** = 75th percentile
* **IQR** = `Q3 - Q1`
```

The lower and upper bounds are calculated as:

Lower Bound = $Q1 - 1.5 \times IQR$

Upper Bound = $Q3 + 1.5 \times IQR$

Any value below the lower bound or above the upper bound is considered an ****outlier****.

```
import pandas as pd
```

```
import seaborn as sns
```

Load Titanic dataset

```
df = sns.load_dataset("titanic")
```

Calculate Q1, Q3, and IQR

```
Q1 = df['fare'].quantile(0.25)
```

```
Q3 = df['fare'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

Calculate lower and upper bounds

```
lower_bound = Q1 - 1.5 * IQR
```

```

upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

Identify outliers
outliers = df[(df['fare'] < lower_bound) | (df['fare'] > upper_bound)]

print("Number of outliers:", len(outliers))
print("Outliers:")
print(outliers['fare'])

Cap (Winsorize) the outliers
df['fare'] = df['fare'].clip(lower=lower_bound, upper=upper_bound)

print("Outliers have been capped.")

```

The program first calculates **Q1, Q3, and IQR** for the `'fare'` column. It then identifies values below the lower bound or above the upper bound as outliers.

Instead of removing these outliers, the `'clip()'` method **caps** them:

- * Values below the lower bound are changed to the **lower bound**.
- * Values above the upper bound are changed to the **upper bound**.

This process is called **Winsorization**, and it keeps all rows in the dataset while limiting the effect of extreme values.

Question 10: You are preparing the Titanic dataset for a machine learning model. Write a Python program that: (a) label-encodes the binary 'sex' column, (b) one-hot encodes the 'embarked' column using `pd.get_dummies()`, and (c) standardizes the 'fare' column using Z-score scaling ($x_scaled = (x - \text{mean}) / \text{std}$). Explain why encoding and scaling are necessary before feeding data into most ML models.

-->

Encoding and scaling are important steps in preparing data for machine learning models. ****Encoding**** converts categorical data into numerical form, while ****scaling**** puts numerical features on a similar scale so that features with larger values do not dominate the model.

```
import pandas as pd
import seaborn as sns

# Load Titanic dataset
df = sns.load_dataset("titanic")

# (a) Label encode the binary 'sex' column
df['sex_encoded'] = df['sex'].map({'male': 0, 'female': 1})

# (b) One-hot encode the 'embarked' column
df = pd.get_dummies(df, columns=['embarked'], dtype=int)

# (c) Standardize the 'fare' column using Z-score scaling
mean = df['fare'].mean()
std = df['fare'].std()

df['fare_scaled'] = (df['fare'] - mean) / std

# Display the required columns
print(df[['sex', 'sex_encoded', 'fare', 'fare_scaled',
          'embarked_C', 'embarked_Q', 'embarked_S']].head())
```

****1. Label Encoding - `sex`****

The ``sex`` column contains two categories: ``male`` and ``female``. We convert them into numerical values:

```
male    → 0
female  → 1
```

This allows a machine learning model to process the categorical information numerically.

****2. One-Hot Encoding - `embarked`****

The `embarked` column contains categories such as `C`, `Q`, and `S`.

`pd.get\_dummies()` creates separate binary columns:

```
embarked_C  
embarked_Q  
embarked_S
```

Each column contains `1` if the passenger belongs to that category and `0` otherwise.

****3. Z-score Scaling - `fare`****

The `fare` column is standardized using:

```
x_scaled = (x - mean) / std
```

After scaling, the values are centered around ****0**** with a standard deviation of approximately ****1****.

.